Cycle Sort

> 💡 Cycle Sort is an in-place, **unstable**, comparison-based sorting algorithm that is optimal in terms of the minimum number of memory writes.

Unstable sorting algorithm can be defined as **sorting algorithm** in which the order of objects with the same values in the sorted array are not guaranteed to be the same as in the input array.

Some examples of unstable sorting algorithms are:

- **Selection Sort**
- **Quick Sort**
- **Heap Sort**

The Core Idea

- For every element, count how many elements are smaller than it.
- This count gives its correct position in the sorted array.
- If the element is not already at its correct position, place it there.
- While placing, another element gets displaced -repeat the same process for that element.
- This continues until we come back to the starting point, forming a cycle.

What the Code Does — Theoretically

Outer loop — `for cycle_start in range(0, n-1)`

This moves through the array **starting a new cycle at each position**. Think of it as saying:

> "Let me make sure the element at this position is correctly placed, and along the way place everyone else in that cycle too."

Once a cycle is complete, everything involved in that cycle is in its correct place — **forever**. You never touch them again.

Finding the correct position

```
pos = cycle_start
for i in range(cycle_start + 1, n):
    if arr[i] < item:
        pos += 1
```

This answers: **"where does this element belong?"**

The correct index of any element = number of elements smaller than it. So you scan the rest of the array and count how many are smaller. That count is its rightful index.

```
item = 3, rest of array = [1, 4, 2]
elements smaller than 3 → {1, 2} → 2 elements
so 3 belongs at index 2
```

Early exit — `if pos == cycle_start`

```
if pos == cycle_start:
    continue
```

If the correct position is where it already sits — **it's already home**. No cycle needed, move on.

Skipping duplicates

```
while item == arr[pos]:
    pos += 1
```

If the element at the target position is the same value as item, placing it there changes nothing. So nudge `pos` forward until you find a different value to displace.

Placing the element and picking up the displaced one

```
arr[pos], item = item, arr[pos]
```

This is the heart of cycle sort. You don't do a full swap — you:

- **Place** item at its correct position
- **Pick up** whoever was sitting there
- Now `item` holds the newly displaced element that also needs to find its home

Inner while loop — following the cycle

```
while pos != cycle_start:
    # recalculate pos for new item
    # skip duplicates
    # place item, pick up displaced element
```

This keeps repeating the same logic — find position, place, pick up — for each newly displaced element. The cycle is complete when the displaced element belongs back at `cycle_start`. That means the chain has closed:

```
A displaced B
B displaced C
C displaced A  ← back to start, cycle closed
```

The Full Picture

```
Phase 1 — Find:    Where does this element belong?
Phase 2 — Place:   Put it there, pick up the displaced elemen
t
```

```
Phase 3 — Repeat:  Keep placing displaced elements until cycl
e closes
Phase 4 — Move on: Start a new cycle at the next index
```

The genius of cycle sort is that **every element is placed exactly once** — you never move an element unless you're putting it in its final position. That's why it achieves the theoretical minimum number of writes.

Why is this useful for First Missing Positive?

After cycle sort, the array looks like:

```
index: 0  1  2  3
value: 1  2  3  4   ← perfect world

index: 0  1  2  3
value: 1  2  4  4   ← 3 is missing, index 2 has wrong value
```

You just scan for the **first index where** `arr[i] != i+1` — that's your missing number. Clean, simple, no negative marking tricks needed.

Cycle Sort With Range

What "range 1 to N" means here

It means the **values** in the array are guaranteed to be between 1 and N, where N is the length of the array.

```
arr = [3, 1, 4, 2]   n = 4
values: 3, 1, 4, 2 → all fall within [1..4] ✅ special case a
pplies
```

Counter examples where it does NOT apply

```
arr = [3, 4, -1, 1]   n = 4
values: 3, 4, -1, 1 → -1 is outside [1..4] ❌

arr = [7, 8, 9, 11, 12]   n = 5
values: 7,8,9,11,12 → all outside [1..5] ❌

arr = [1, 2, 6, 4]   n = 4
values: 1,2,6,4 → 6 is outside [1..4] ❌
```

These are exactly the kinds of arrays you see in **First Missing Positive** — values can be anything, so the special case does NOT directly apply, which is why you needed the extra sanitization steps.

Why the special case is powerful

When values ARE in range `[1..N]`, you already know the exact index each element belongs to **without counting**:

```python
# General cycle sort — must COUNT to find position:
pos = cycle_start
for i in range(cycle_start + 1, n):
    if arr[i] < item:
        pos += 1          # O(n) per element

# Special case — position is directly calculated:
correct_index = value - 1  # O(1) per element
```

So instead of an inner loop to find the position, you just do:

```python
while nums[i] != i + 1:                    # is it in the rig
ht place?
    correct = nums[i] - 1                   # correct index =
value - 1
    nums[i], nums[correct] = nums[correct], nums[i]   # swap
it there
```

This is why the First Missing Positive solution using this optimized cycle sort looks much cleaner than general cycle sort.

Summary

	General Arrays	Range [1..N] Arrays
Example	`[3, -1, 6, 2]`	`[3, 1, 4, 2]`
Find correct index	Count smaller elements — `O(n)`	`value - 1` — `O(1)`
Overall complexity	`O(n²)`	`O(n)`
Cycle detection needed	Yes	No — just swap directly

Without Range Constraint — General Cycle Sort

Why: Minimizes the number of **memory writes** (not comparisons). Every element is written to its final position exactly once.

Use it when: Writing is expensive — like writing to a **flash drive / SSD** where each write degrades the hardware, or writing to a **database** where each write is costly.

```
Goal: sort with minimum writes
Cost: O(n²) time, O(1) space, O(n) writes
```

With Range Constraint [1..N] — Optimized Cycle Sort

Why: When values are in range `[1..N]`, each element's correct index is **directly known** (`value - 1`), so no inner counting loop is needed.

Use it when: You need to find **missing / duplicate / misplaced numbers** in an array where values are constrained to a known range.

```
Goal: place each number at its correct index in O(n)
Use for: First Missing Positive, Find the Duplicate,
         Find all Duplicates, Missing Number
```

One Line Each

	One Line
General	"Sort with minimum writes when writing is expensive"
Range [1..N]	"Use the array as a map — each value tells you exactly where it belongs"